Zealand

Fag: **Udvidet netværksteknologi -** Sune Andersen (SUAN).

Opgave 5 – MQTT IoT Lab – Secure Smart Building

Case

Nordic Smart Office vil overvåge et mødelokale ved hjælp af IoT-sensorer. Sensorerne skal sende **temperatur, luftfugtighed og CO_2** til en central MQTT-server.

NB! I har ingen fysiske IoT-enheder i dette fag. Derfor simulerer I sensorerne med **Python**.

Jeres løsning

Python Sensor -> MQTT -> Mosquitto Broker -> MQTT -> Grafana

Bemærk : Authentication og ACL – da jeres cloud er på det ”store net”

MQTT – få kommunikationen til at virke

På Windows / Linux : Åbn to terminaler.

Terminal 1 – subscriber: mosquitto_sub -h localhost -t "test/message"

Terminal 2 – publisher: mosquitto_pub -h localhost -t "test/message" -m "Hello MQTT"

Test Prøv at ændre: topic – besked - subscriber

Spørgsmål: Hvad er forskellen på en MQTT **publisher** og **subscriber**?

Log ind på jeres Azure Ubuntu-server og installer Mosquitto samt MQTT-klienterne.

Udskift nu localhost med jeres Azure-IP

MQTT Topics : Brug følgende topic-struktur:

building/floor1/room101/temperature

building/floor1/room101/humidity

building/floor1/room101/co2

Send eksempelvis:

mosquitto_pub -h **Azure-IP** -t "building/floor1/room101/temperature" -m "22.5"

Abonnér på alle data fra Room 101: mosquitto_sub -h **Azure-IP** -t "building/floor1/room101/#"

Prøv også: building/+/room101/# **Undersøg:** Hvad betyder + og # i MQTT?

Lav en IoT-sensor med Python

Opret: ***sensor.py***

Sensoren skal: forbinde til MQTT-brokeren

generere temperatur, luftfugtighed og CO_2

sende data hvert 5. sekund

sende data som JSON

bruge Room 101

Brug topic: building/floor1/room101/sensors

Eksempel på payload:

```
{
  "temperature": 22.4,
  "humidity": 45.2,
  "co2": 612
}
```

Kør: python3 sensor.py (se kode eksempel)

Kontrollér med: mosquitto_sub -h* Azure-IP *-t "building/floor1/room101/sensors"

Lav en MQTT Monitor Opret: monitor.py

Programmet skal abonnere på: building/+/+/sensors

Det skal vise sensordata på en overskuelig måde, fx:

Room 101 + Temperature: 22.4 °C + Humidity: 45.2 % + CO2: 612 ppm

Kør sensor og monitor samtidig.

Test : Prøv at ændre sensordataene. Prøv eventuelt at simulere flere rooms ved at ændre topic.

Sikkerhed i MQTT Indtil nu kan alle, der kan nå MQTT-brokeren, forsøge at kommunikere med den.

Nu skal I sikre løsningen.

Authentication

Opret en MQTT-bruger: iotuser

MQTT skal nu kræve: Username + Password

Test både med korrekt og forkert login.

Kontrollér: Kan en bruger uden korrekt password sende eller modtage data?

ACL – hvem må hvad?

Opret mindst to roller: iot-sensor og iot-monitor

Sensoren skal kunne: PUBLISH

building/floor1/room101/sensors

Monitoren skal kunne: SUBSCRIBE

building/floor1/room101/sensors

Test derefter: Sensor sender data. <-> Monitor modtager data.

Sensor forsøger at skrive til et andet topic.

Monitor forsøger at læse et topic, den ikke har adgang til.

Forklar: Hvad er forskellen på **authentication** og **authorization**?

Lav en CO_2-alarm Tilføj en simpel alarm til monitor.py.

If CO2 > 1000 ppm

print WARNING: HIGH CO2 LEVEL

if CO2 > 1500 ppm

print ALARM: CRITICAL CO2 LEVEL

Test ved at ændre den simulerede CO_2-værdi.

Overvej: Hvorfor kan QoS og hurtig kommunikation være vigtigt for en alarm?

TLS og sikker kommunikation Undersøg, hvordan MQTT kan sikres med **TLS**.

Sammenlign: MQTT vs MQTT + Username/Password vs MQTT + TLS

Kode eksempel til sensor.py

```
import json
import paho.mqtt.client as mqtt
BROKER = "localhost"
PORT = 1883
TOPIC = "building/+/+/sensors"
def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print("Forbundet til MQTT broker")
        client.subscribe(TOPIC)
        print(f"Subscriber på: {TOPIC}")
    else:
        print(f"Fejl ved forbindelse: {reason_code}")
def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        print("\n--- IoT DATA ---")
        print(f"Topic:       {msg.topic}")
        print(f"Temperature: {data['temperature']} °C")
        print(f"Humidity:    {data['humidity']} %")
        print(f"CO2:         {data['co2']} ppm")
        # CO2 alarm
        if data["co2"] > 1500:
            print("!!! ALARM: CRITICAL CO2 LEVEL !!!")
        elif data["co2"] > 1000:
            print("WARNING: HIGH CO2 LEVEL")

    except json.JSONDecodeError:
        print("Fejl: Modtog ikke gyldig JSON")

    except KeyError as e:
        print(f"Fejl: Manglende datafelt: {e}")
client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="room101-monitor"
)

client.on_connect = on_connect
client.on_message = on_message
print("Starter MQTT Monitor...")
client.connect(BROKER, PORT, 60)
client.loop_forever()
```

Kode eksempel til: monitor.py

```
import json
import paho.mqtt.client as mqtt
# MQTT broker
BROKER = "localhost"
PORT = 1883
# MQTT topic
TOPIC = "building/+/+/sensors"
# Når forbindelsen til MQTT broker er oprettet
def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print("Forbundet til MQTT broker")
        client.subscribe(TOPIC)
        print(f"Subscriber på: {TOPIC}")
    else:
        print(f"Kunne ikke forbinde: {reason_code}")
# Når der modtages en MQTT besked
def on_message(client, userdata, msg):
    try:
        # Konverter MQTT payload fra tekst til JSON
        data = json.loads(msg.payload.decode())
        print("\n--- IoT DATA ---")
        print(f"Topic:       {msg.topic}")
        print(f"Temperature: {data['temperature']} °C")
        print(f"Humidity:    {data['humidity']} %")
        print(f"CO2:         {data['co2']} ppm")
  # CO2 alarm
        if data["co2"] > 1500:
            print("!!! ALARM: CRITICAL CO2 LEVEL !!!")
        elif data["co2"] > 1000:
            print("WARNING: HIGH CO2 LEVEL")
    except json.JSONDecodeError:
        print("Fejl: Modtaget data er ikke gyldig JSON")
    except KeyError as error:
        print(f"Fejl: Manglende datafelt: {error}")
# Opret MQTT client
client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2,
    client_id="room101-monitor"
)

# Registrer funktioner
client.on_connect = on_connect
client.on_message = on_message
# Forbind til MQTT broker
print("Starter MQTT Monitor...")
client.connect(BROKER, PORT, 60)

# Hold forbindelsen kørende
client.loop_forever()
```